UNIVERSITY OF
WEST LONDON

Project Title

NorthStar Urban Mobility Analytics Project

Submitted By: Gabriel Jackson

Student number: 33104535

Course: Information Technology

Module: Databases and Analytics

Supervisor: Shidrokh Goudarzi

Submission Date: 12 May 2026 23:59:00 BST

Table of Contents

Introduction

NorthStar Urban Mobility and Logistics is a fast-growing organisation operating across multiple UK cities, offering services such as public transport, last-mile delivery, warehouse dispatch, electric vehicle charging, and mobile platform services.

The rapid growth and integration of multiple systems have presented the organisation with considerable operational and analytical challenges.

Every day, enormous quantities of structured and semi-structured data are generated from customer transactions, vehicle operations, route-planning systems, complaint records, driver applications, and mobile platform interactions.

But these data sources remain incoherent and difficult to analyse well.

The project is about developing an integrated analytics Solution using Python, R, SQL, and MongoDB Atlas to enable enhanced operational visibility and decision-making.

Data cleaning and preparation were done using Python and SQL. Structured queries and business-oriented analysis were performed using R.

We applied R analytics and visualisation techniques to the organisation's data to identify trends, inefficiencies and operational patterns.

MongoDB Atlas was introduced as a NoSQL database solution to show that flexible and nested operational records can be managed better than with relational structures(MongoDB Inc., 2025).

The Main objective of this Project is to identify the key operational problems that impact NorthStar. These problems include delivery failures, customer complaints, inefficient route allocation and inconsistent service performance.

This project shows how data analytics and modern database systems can be used to enable more effective operational management and data-driven decision-making by integrating relational and NoSQL technologies in a single workflow. The notebooks are arranged as a full analytics pipeline.

It begins with cleaning and standardising raw operational data, then moves into relational querying and business analysis in R, followed by visual reporting with an insight focus, and finally a MongoDB implementation demonstrating NoSQL modelling and database operations.

Data Cleaning

Notebook 1: 1-NorthStar_Data_Cleanup.ipynb

The first notebook handles data preparation in Python and acts as the base for everything that follows. It loads the project datasets (hubs, customers, drivers, vehicles, orders, deliveries, incidents, complaints, and app_events), then performs foundational cleaning steps. The notebook standardises zone fields, applies string normalisation, and converts important time-based columns into a proper datetime format so later analysis can run reliably.

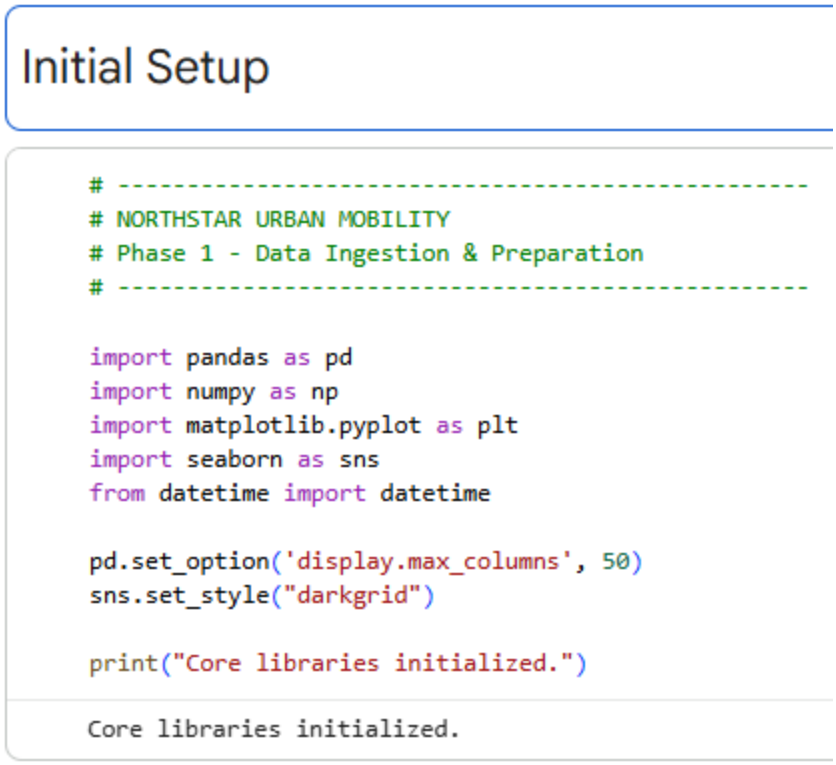

Figure 1: Initial Python Environment and Library Configuration

Figure 1 shows the initial setup phase of the analytics workflow, including importing and configuring core Python libraries such as Pandas, NumPy, Matplotlib, and Seaborn. These libraries were used throughout the project for data preparation, analysis, and visualisation tasks.

Data Loading

```
# ------------------------------------------------
# Loading Source CSV Files
# ------------------------------------------------

source_files = {
    'hubs': 'hubs.csv',
    'customers': 'customers.csv',
    'drivers': 'drivers.csv',
    'vehicles': 'vehicles.csv',
    'orders': 'orders.csv',
    'deliveries': 'deliveries.csv',
    'incidents': 'incidents.csv',
    'complaints': 'complaints.csv',
    'app_events': 'app_events.csv'
}

source_data = {}
for label, fname in source_files.items():
    try:
        source_data[label] = pd.read_csv(fname)
        print(f"Imported {label:12} | Rows: {source_data[label].shape[0]:,}")
    except Exception as err:
        print(f"Could not load {fname} → {err}")
```

```
Imported hubs         | Rows: 8
Imported customers    | Rows: 650
Imported drivers      | Rows: 170
Imported vehicles     | Rows: 120
Imported orders       | Rows: 1,250
Imported deliveries   | Rows: 950
Imported incidents    | Rows: 280
Imported complaints   | Rows: 320
Imported app_events   | Rows: 640
```

Figure 2: Raw NorthStar operational datasets loaded into Python

This figure shows the initial data loading stage, in which multiple raw CSV datasets were imported into Python using the pandas library. The datasets include operational data such as customers, drivers, deliveries, orders, incidents, complaints, and app events.

The successful import of all operational datasets confirms that the data collection process was completed correctly and that the datasets were ready for preprocessing and analysis. This step forms the foundation for subsequent cleaning, SQL querying, data visualisation, and MongoDB implementation tasks carried out in the project.

Dataset Cleaning

```
# ------------------------------------------------
# Zone Name Standardization & Basic Cleaning
# ------------------------------------------------

def clean_zone(val):
    if pd.isna(val):
        return val
    cleaned = str(val).strip().title()
    cleaned = cleaned.replace("Riverside", "Riverside").replace("RiverSide", "Riverside")
    cleaned = cleaned.replace("Ctr", "Central")
    return cleaned

for key in ['orders', 'deliveries', 'customers', 'drivers', 'hubs']:
    if key in source_data:
        df = source_data[key]
        for col in df.columns:
            if 'zone' in col.lower():
                df[col] = df[col].apply(clean_zone)
        print(f"Zone standardization completed for {key}")
```

```
Zone standardization completed for orders
Zone standardization completed for deliveries
Zone standardization completed for customers
Zone standardization completed for drivers
Zone standardization completed for hubs
```

Figure 3: Data cleaning and datetime standardisation process

This figure conveys the cleaning process used in Python to fix inconsistent zone names and format issues across the datasets. The script standardises inconsistent zone names and applies cleaning functions across multiple operational datasets to improve data consistency and quality. The cleaning process improved dataset reliability by removing inconsistencies in naming conventions and formatting. Standardised data ensures more accurate querying, analysis, and reporting throughout the project workflow.

Temporal Data Handling

```
# ------------------------------------------------
# Date Parsing Across Tables
# ------------------------------------------------

date_fields = {
    'orders': ['order_created_at'],
    'deliveries': ['dispatch_time', 'delivery_completed_at'],
    'complaints': ['created_at'],
    'app_events': ['event_timestamp'],
    'incidents': ['reported_at']
}

for table_key, cols in date_fields.items():
    if table_key in source_data:
        df = source_data[table_key]
        for c in cols:
            if c in df.columns:
                df[c] = pd.to_datetime(df[c], errors='coerce')
                print(f"Date parsing done → {table_key}.{c}")
```

```
Date parsing done → orders.order_created_at
Date parsing done → deliveries.dispatch_time
Date parsing done → deliveries.delivery_completed_at
Date parsing done → complaints.created_at
Date parsing done → app_events.event_timestamp
Date parsing done → incidents.reported_at
```

Figure 4: Temporal Data Handling and Date Parsing

This figure shows the implementation of temporal data handling using Python datetime parsing functions. The process converts multiple date-related fields into consistent datetime formats across operational tables.

Converting the date columns into a datetime format made it easier to analyse delivery times, complaint trends, and incident timings, including delivery tracking, incident monitoring, and complaint trend evaluation. Consistent datetime formatting also improves compatibility with SQL and MongoDB operations.

Export Cleaned Files

```
# ------------------------------------------------
# Exporting Final Cleaned Version
# ------------------------------------------------

import os
os.makedirs('northstar_ready', exist_ok=True)

for name, df in source_data.items():
    target_file = f'northstar_ready/{name}_ready.csv'
    df.to_csv(target_file, index=False)
    print(f"Exported → {target_file}")
```

```
Exported → northstar_ready/hubs_ready.csv
Exported → northstar_ready/customers_ready.csv
Exported → northstar_ready/drivers_ready.csv
Exported → northstar_ready/vehicles_ready.csv
Exported → northstar_ready/orders_ready.csv
Exported → northstar_ready/deliveries_ready.csv
Exported → northstar_ready/incidents_ready.csv
Exported → northstar_ready/complaints_ready.csv
Exported → northstar_ready/app_events_ready.csv
```

Figure 5: Export of cleaned datasets into analysis-ready CSV files

After cleaning, it exports a set of processed files to the northstar_cleaned_data folder using the *_ready.csv naming convention. Also, it packages those outputs into a zip archive for transfer or reuse. In short, this notebook turns raw CSV inputs into analysis-ready data.

The export process ensured that all cleaned datasets were preserved in a structured format for future SQL analysis, visualisation, and MongoDB implementation. This step improved workflow efficiency by separating raw data from validated analysis-ready data.

SQL in R Analysis

Notebook 2: 2-NorthStar_R_Querying.ipynb

The second notebook takes the cleaned outputs and shifts to R for SQL-style analysis. It loads required R packages, imports the processed CSV files, and writes them into a SQLite database (northstar_query.db) as structured tables. Once the database is ready, it runs business-oriented SQL queries like high-value customer analysis, vehicle utilisation with incident context, time-of-day delivery performance and Driver experience versus outcomes.

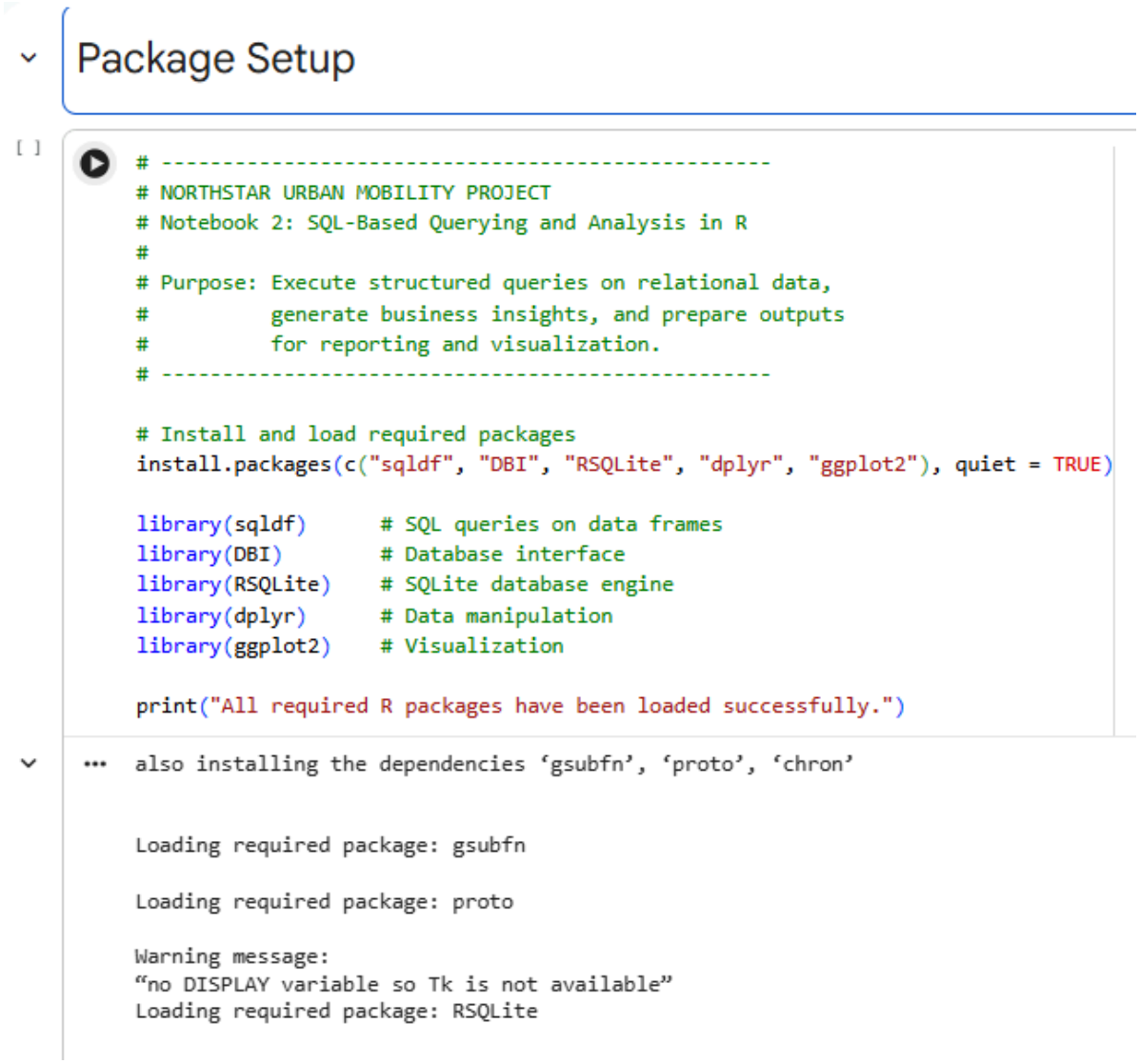

Figure 6: SQLite database tables created from cleaned datasets

This figure illustrates the setup of the SQLite analytical environment in R, including installing required packages and preparing cleaned datasets for relational database operations.

After importing the cleaned datasets into SQLite, SQL queries could be used more efficiently for structured querying and analysis. This relational layer supports efficient SQL-based analysis and reporting.

Data Import

```r
# ------------------------------------------------
# Import Processed Datasets
#
# Note: Files must be the output from Notebook 1
# (northstar_ready folder)
# ------------------------------------------------

processed_items <- c("complaints", "deliveries", "orders", "vehicles",
                     "drivers", "customers", "incidents", "app_events", "hubs")

for (item in processed_items) {
  df_name <- paste0(item, "_ready")
  file_path <- paste0(item, "_ready.csv")

  if (file.exists(file_path)) {
    assign(df_name, read.csv(file_path, stringsAsFactors = FALSE))
    cat("Successfully imported:", df_name, "→", nrow(get(df_name)), "records\n")
  } else {
    cat("Warning: File not found -", file_path, "\n")
  }
}

print("Data import phase completed.")
```

```
Successfully imported: complaints_ready → 320 records
Successfully imported: deliveries_ready → 950 records
Successfully imported: orders_ready → 1250 records
Successfully imported: vehicles_ready → 120 records
Successfully imported: drivers_ready → 170 records
Successfully imported: customers_ready → 650 records
Successfully imported: incidents_ready → 280 records
Successfully imported: app_events_ready → 640 records
Successfully imported: hubs_ready → 8 records
[1] "Data import phase completed."
```

Figure 7: Import of Cleaned Operational Datasets into R

This figure demonstrates the import of cleaned NorthStar operational datasets into the R environment. The process loads multiple CSV datasets, including complaints,

deliveries, customers, incidents, and operational records, to prepare the data for SQL querying and analysis.

SQLite Database Creation

```
# ------------------------------------------------
# Initialize SQLite Database
#
# All tables are loaded into an in-memory SQLite database
# for efficient SQL querying.
# ------------------------------------------------

conn <- dbConnect(RSQLite::SQLite(), "northstar_query.db")

# Get all data frames ending with _ready
ready_tables <- ls(pattern = "_ready$")

for (tbl in ready_tables) {
  df_object <- get(tbl)
  clean_table_name <- gsub("_ready", "", tbl)

  dbWriteTable(conn, clean_table_name, df_object, overwrite = TRUE, row.names = FALSE)
  cat("Table loaded into database:", clean_table_name, "\n")
}

print("SQLite database is now ready for analysis.")
```

```
Table loaded into database: app_events
Table loaded into database: complaints
Table loaded into database: customers
Table loaded into database: deliveries
Table loaded into database: drivers
Table loaded into database: hubs
Table loaded into database: incidents
Table loaded into database: orders
Table loaded into database: vehicles
[1] "SQLite database is now ready for analysis."
```

Figure 8: SQLite Database Initialisation and Table Loading

This figure illustrates the creation of the SQLite analytical database used within the project. Cleaned datasets were loaded into an in-memory SQLite database to support efficient SQL querying, relational analysis, and business intelligence operations.

The successful database initialisation demonstrates how relational database systems can efficiently organise operational data for business intelligence purposes. Loading the datasets into SQLite enabled structured querying, relationship analysis, and faster access to analytical results throughout the project.

Core Business Queries

```
# ------------------------------------------------
# Core Business Intelligence Queries
# ------------------------------------------------

cat("=== Query A: High-Value Customer Analysis ===\n")
dbGetQuery(conn, "
  SELECT c.customer_id,
         COUNT(DISTINCT o.order_id) AS total_orders,
         ROUND(SUM(o.order_value), 2) AS total_spent,
         COUNT(comp.complaint_id) AS complaints
  FROM customers c
  JOIN orders o ON c.customer_id = o.customer_id
  LEFT JOIN complaints comp ON o.order_id = comp.order_id
  GROUP BY c.customer_id
  HAVING total_spent > 500
  ORDER BY total_spent DESC
  LIMIT 10
")

cat("\n=== Query B: Vehicle Utilization & Issues ===\n")
dbGetQuery(conn, "
  SELECT v.vehicle_id,
         v.vehicle_type,
         COUNT(DISTINCT d.delivery_id) AS deliveries_made,
         COUNT(i.incident_id) AS incidents,
         ROUND(AVG(v.battery_health_pct), 2) AS avg_battery_health
  FROM vehicles v
  LEFT JOIN deliveries d ON v.vehicle_id = d.vehicle_id
  LEFT JOIN incidents i ON d.delivery_id = i.delivery_id
  GROUP BY v.vehicle_id, v.vehicle_type
  ORDER BY incidents DESC
  LIMIT 8
")
```

Figure 9: SQL-Based Core Business Intelligence Queries

This figure presents a series of SQL business intelligence queries used to analyse high-value customers, delivery activity, complaint frequency, and vehicle utilisation. The queries demonstrate the integration of SQL within R for operational analytics and business-focused reporting.

The SQL queries demonstrate the practical use of relational analysis in identifying high-value customers, operational inefficiencies, and service performance trends. The results help identify customer trends, delivery issues, and vehicle activity patterns.

```
=== Query A: High-Value Customer Analysis ===
```

A data.frame: 10 × 4

customer_id	total_orders	total_spent	complaints
<chr>	<int>	<dbl>	<int>
C0545	6	1433.51	3
C0622	6	732.93	2
C0157	4	720.04	2
C0343	7	685.78	1
C0372	6	669.11	3
C0013	5	659.26	1
C0172	4	625.45	3
C0558	4	605.85	1
C0289	4	601.81	0
C0301	3	592.99	0

```
=== Query B: Vehicle Utilization & Issues ===
```

A data.frame: 8 × 5

vehicle_id	vehicle_type	deliveries_made	incidents	avg_battery_health
<chr>	<chr>	<int>	<int>	<dbl>
V047	EV	17	9	93.7
V108	Diesel	9	7	54.6
V030	CargoVan	15	6	78.0
V046	EV	10	6	95.8
V097	EV	8	6	92.1

Figure 10: SQL query analysing high-value customers

This figure presents the output of SQL queries used to identify high-value customers and analyse vehicle utilisation metrics. The query results include customer spending behaviour, complaint counts, vehicle delivery activity, and operational incidents.

The SQL queries helped identify high-spending customers and operational issues affecting deliveries. These findings support Data Driven decision-making, customer management, and operational optimisation strategies.

Data Visualisation

Visualizations

```
# ------------------------------------------------
# Exploratory Visualizations
# ------------------------------------------------

# 1. Complaint Count by Type
complaint_counts <- complaints_ready %>%
  count(complaint_type) %>%
  arrange(desc(n))

ggplot(complaint_counts, aes(x = reorder(complaint_type, n), y = n)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  coord_flip() +
  labs(title = "Number of Complaints by Type",
       x = "Complaint Type",
       y = "Count") +
  theme_minimal()

# 2. Delivery Status Breakdown
ggplot(deliveries_ready, aes(x = delivery_status)) +
  geom_bar(fill = "darkorange") +
  labs(title = "Delivery Status Distribution",
       x = "Status",
       y = "Number of Deliveries") +
  theme_minimal()

# 3. Problem Rate by Zone (Simple Bar)
zone_summary <- orders_ready %>%
  left_join(deliveries_ready, by = "order_id") %>%
  group_by(pickup_zone) %>%
  summarise(
    total = n(),
    problems = sum(delivery_status %in% c("Delayed", "Failed"))
  ) %>%
  mutate(problem_rate = problems / total * 100)

ggplot(zone_summary, aes(x = reorder(pickup_zone, problem_rate), y = problem_rate)) +
  geom_bar(stat = "identity", fill = "tomato") +
  coord_flip() +
```

Figure 11: Exploratory Data Visualisation and Operational Analysis

The chart revealed the implementation of exploratory visualisation techniques in R using ggplot2. The visualisations analyse complaint distributions, Delivery status breakdowns, and operational problem rates across different service zones to identify performance trends and inefficiencies.

Lastly, the charts made it easier to compare complaint levels, delivery outcomes, and problem areas between zones. They also helped identify recurring service issues and performance differences between delivery zones. These insights support better operational planning, resource allocation, and service quality improvements within the NorthStar Urban Mobility project.

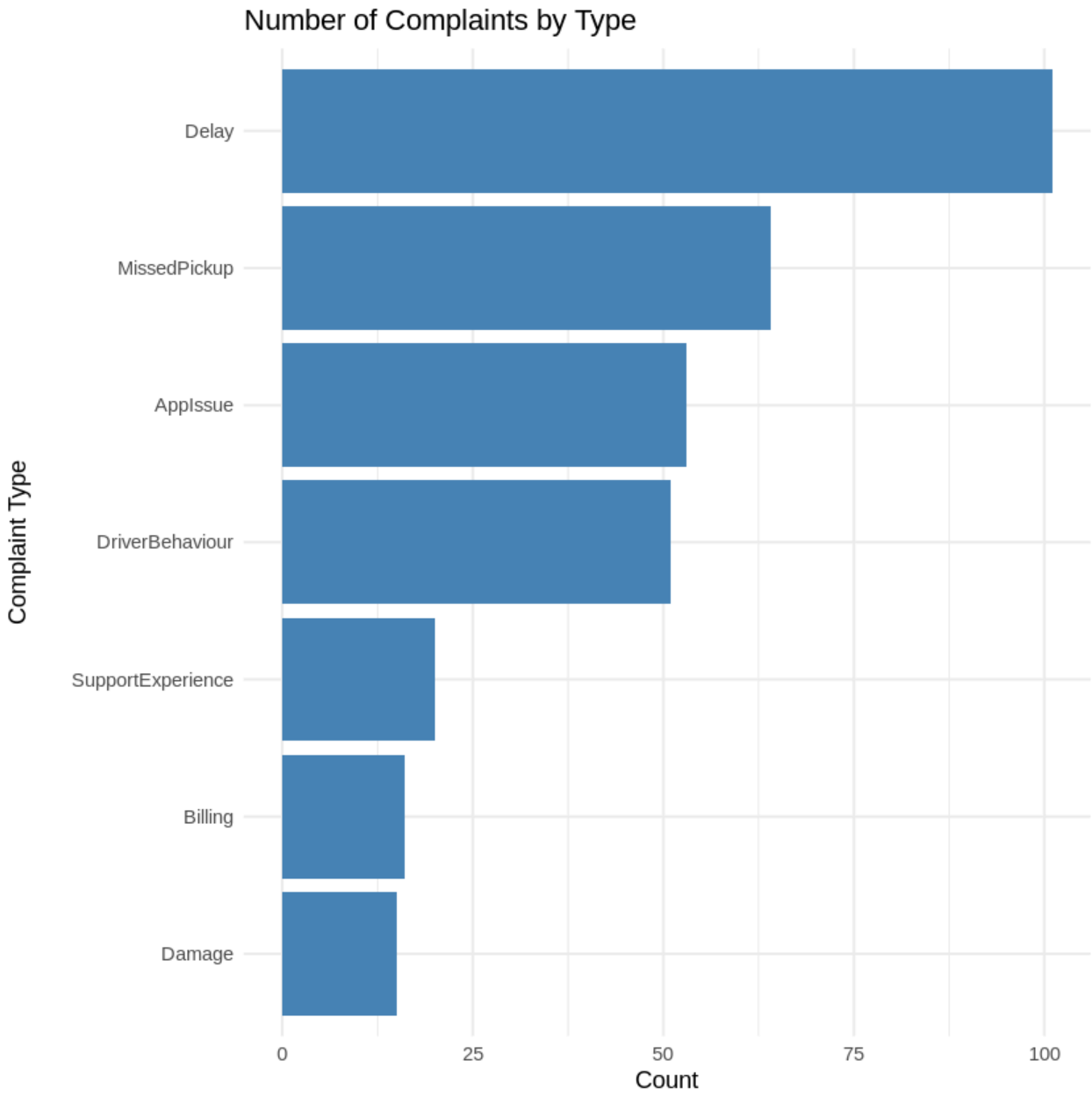

Figure 12: Most common categories of customer complaints

This visualisation shows the frequency of customer complaint categories recorded in the NorthStar operational dataset.

The most common were delays in delivery, then missed pick-ups, and problems with the application. The chart helps operational analysis, highlighting areas where service improvement and customer support optimisation are needed.

The findings suggest that delivery reliability is the main operational challenge impacting customer satisfaction. The high number of complaints related to delays indicates possible inefficiencies in the management of routes, scheduling or resource allocation.

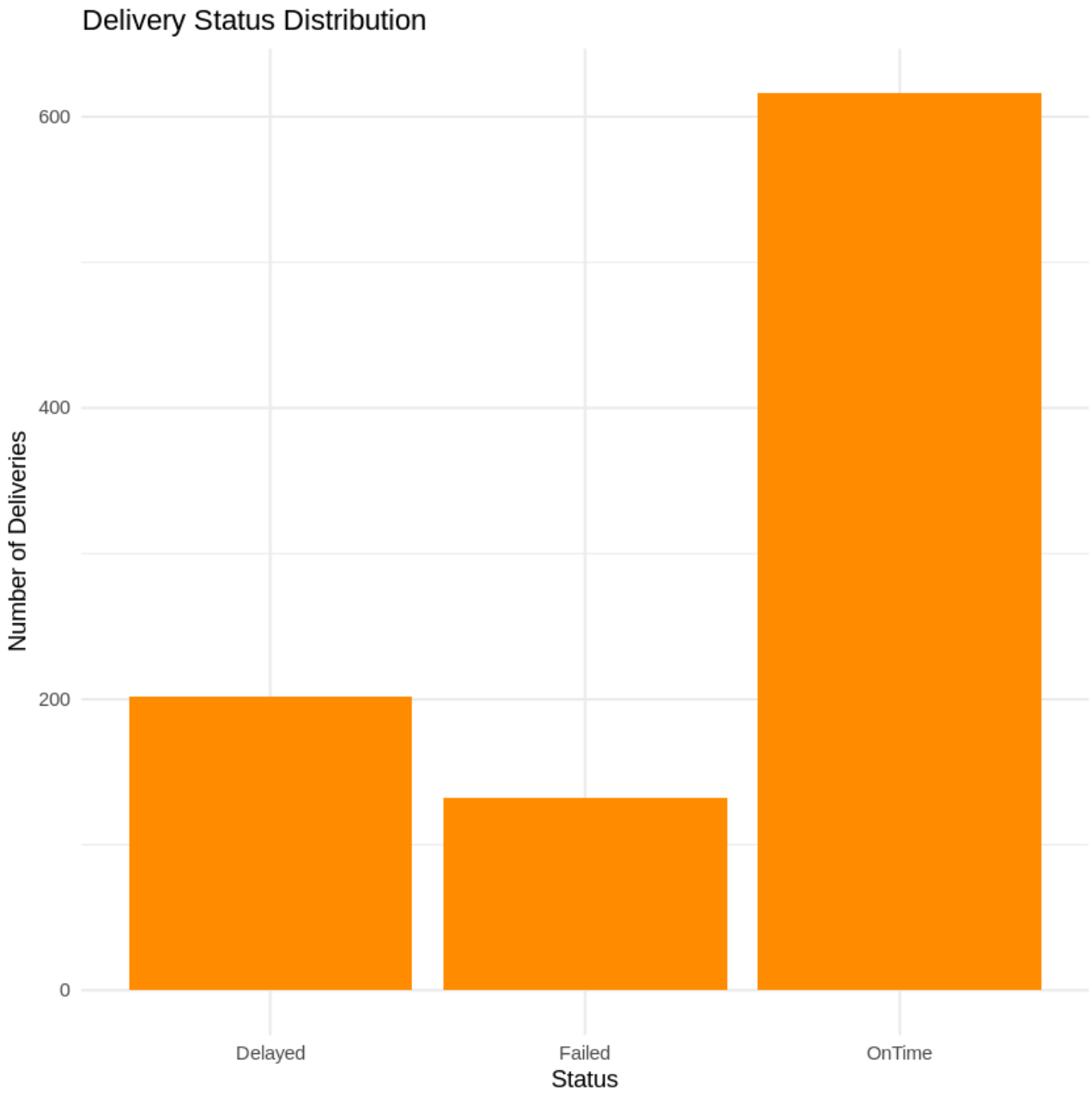

Figure 13: Distribution of delivery outcomes across operational services

This notebook also includes visual summaries in R to support interpretation of those query results, especially around complaint patterns, delivery outcomes, and zone-level operational issues. Functionally, Notebook 2 is the project's relational analytics layer.

Although most deliveries were completed successfully, delayed and failed deliveries highlight areas where operational performance can still be improved. Reducing these failed outcomes could significantly improve customer experience and service consistency.

Notebook 3: 3-NorthStar_R_Insights_Viz.ipynb

The third notebook remains in R but emphasises insight communication more than SQL mechanics. It reloads the cleaned datasets and produces concise KPI-style summaries, including delivery status proportions and the most common complaint categories. It then builds straightforward plots to make those findings easier to explain, including distributions of delivery outcomes, counts of complaint channels, and pickup-zone problem rates.

Setup

```
# ------------------------------------------------
# NORTHSTAR URBAN MOBILITY PROJECT
# Notebook 3: Insights Generation and Visualization
#
# Objective: Perform exploratory data analysis, generate
#            key business insights, and create clear
#            visualizations to support decision making.
# ------------------------------------------------

# Install and load necessary packages
install.packages(c("dplyr", "ggplot2", "tidyr", "lubridate"), quiet = TRUE)

library(dplyr)      # Data manipulation and summarization
library(ggplot2)    # Creating visualizations
library(tidyr)      # Data reshaping
library(lubridate)  # Working with dates

print("All packages loaded successfully for exploratory analysis.")
```

```
Attaching package: ‘lubridate’


The following objects are masked from ‘package:base’:

    date, intersect, setdiff, union


[1] "All packages loaded successfully for exploratory analysis."
```

Figure 14: Initialisation of Exploratory Data Analysis Environment in R

This stage prepared the R environment for exploratory data analysis within the R environment. Furthermore, essential libraries such as dplyr, ggplot2, tidyr, and lubridate were loaded to support data manipulation, visualisation, and analytical reporting throughout the project workflow.

These R libraries were useful for organising the data, creating charts, and analysing operational trends throughout the project.

Load Data

```
# ----------------------------------------------------
# Loading the Prepared Datasets
#
# These files are the output from the data cleanup notebook.
# ----------------------------------------------------

files_list <- c("complaints", "deliveries", "orders", "vehicles",
                "drivers", "customers", "incidents", "app_events", "hubs")

for (file_name in files_list) {
  df_name <- paste0(file_name, "_ready")
  file_path <- paste0(file_name, "_ready.csv")

  if (file.exists(file_path)) {
    assign(df_name, read.csv(file_path, stringsAsFactors = FALSE))
    cat("Successfully loaded:", df_name, "with", nrow(get(df_name)), "records\n")
  } else {
    cat("File not found:", file_path, "\n")
  }
}

print("Data loading phase completed.")
```

```
Successfully loaded: complaints_ready with 320 records
Successfully loaded: deliveries_ready with 950 records
Successfully loaded: orders_ready with 1250 records
Successfully loaded: vehicles_ready with 120 records
Successfully loaded: drivers_ready with 170 records
Successfully loaded: customers_ready with 650 records
Successfully loaded: incidents_ready with 280 records
Successfully loaded: app_events_ready with 640 records
Successfully loaded: hubs_ready with 8 records
[1] "Data loading phase completed."
```

Figure 15: Loading Prepared Operational Datasets for Visual Analytics

Figure 15 shows the process of loading cleaned operational datasets into the R environment for exploratory analysis and visualisation. Multiple datasets, including complaints, deliveries, customers, incidents, and vehicles, were successfully imported to support analytical processing and the generation of business insights.

Successfully importing multiple datasets ensured cross-functional analysis across operational areas and more accurate business intelligence reporting and decision-making.

Basic Insights

```
# ------------------------------------------------
# Generating Summary Statistics
#
# These summaries help understand the overall health of operations.
# ------------------------------------------------

cat("=== Delivery Status Overview ===\n")
deliveries_ready %>%
  count(delivery_status) %>%
  mutate(percentage = round(n / sum(n) * 100, 2)) %>%
  print()

cat("\n=== Most Common Complaint Types ===\n")
complaints_ready %>%
  count(complaint_type, sort = TRUE) %>%
  head(6) %>%
  print()
```

```
=== Delivery Status Overview ===
  delivery_status   n percentage
1         Delayed 202      21.26
2          Failed 132      13.89
3          OnTime 616      64.84

=== Most Common Complaint Types ===
     complaint_type   n
1             Delay 101
2      MissedPickup  64
3          AppIssue  53
4   DriverBehaviour  51
5 SupportExperience  20
6           Billing  16
```

Figure 16: Generation of Operational Summary Statistics

This figure presents summary statistics generated from the delivery and complaint datasets. The analysis identifies the distribution of delivery outcomes and the most common complaint categories, providing an overview of operational performance and customer service trends.

The summary statistics reveal that while most deliveries were completed on time, customer complaints remain concentrated around delivery delays and missed pickups. This suggests that improving operational efficiency could reduce complaint frequency and improve customer satisfaction.

Visualizations

```r
# ------------------------------------------------
# Creating Simple and Clear Visualizations
# ------------------------------------------------

# Chart 1: Delivery Status Distribution
ggplot(deliveries_ready, aes(x = delivery_status)) +
  geom_bar(fill = "skyblue", color = "black") +
  labs(title = "Distribution of Delivery Outcomes",
       subtitle = "Shows the current performance level of delivery operations",
       x = "Delivery Status",
       y = "Number of Deliveries") +
  theme_minimal()

# Chart 2: Complaints by Channel
ggplot(complaints_ready, aes(x = channel)) +
  geom_bar(fill = "salmon", color = "black") +
  labs(title = "Number of Complaints by Communication Channel",
       x = "Channel",
       y = "Complaint Count") +
  theme_minimal() +
  coord_flip()

# Chart 3: Problem Rate by Zone
zone_summary <- orders_ready %>%
  left_join(deliveries_ready, by = "order_id") %>%
  group_by(pickup_zone) %>%
  summarise(
    total_orders = n(),
    problems = sum(delivery_status %in% c("Delayed", "Failed"))
  ) %>%
  mutate(problem_rate = round(problems / total_orders * 100, 1))

ggplot(zone_summary, aes(x = reorder(pickup_zone, problem_rate), y = problem_rate)) +
  geom_col(fill = "tomato", color = "black") +
  labs(title = "Delivery Problem Rate by Pickup Zone",
       subtitle = "Higher bars indicate zones needing attention",
       x = "Pickup Zone",
       y = "Problem Rate (%)") +
  theme_minimal() +
  coord_flip()
```

Figure 17: Development of Operational Performance Visualisations

The chart was created using ggplot2 in R to visualise operational trends and complaint patterns. The code generates charts of delivery status distributions, complaint communication channels, and delivery problem rates across operational zones to help identify operational trends and problem areas.

The visualisation scripts made the operational data easier to understand through visual charts and summaries, also making it easier to identify performance trends, operational risks, and areas requiring managerial attention.

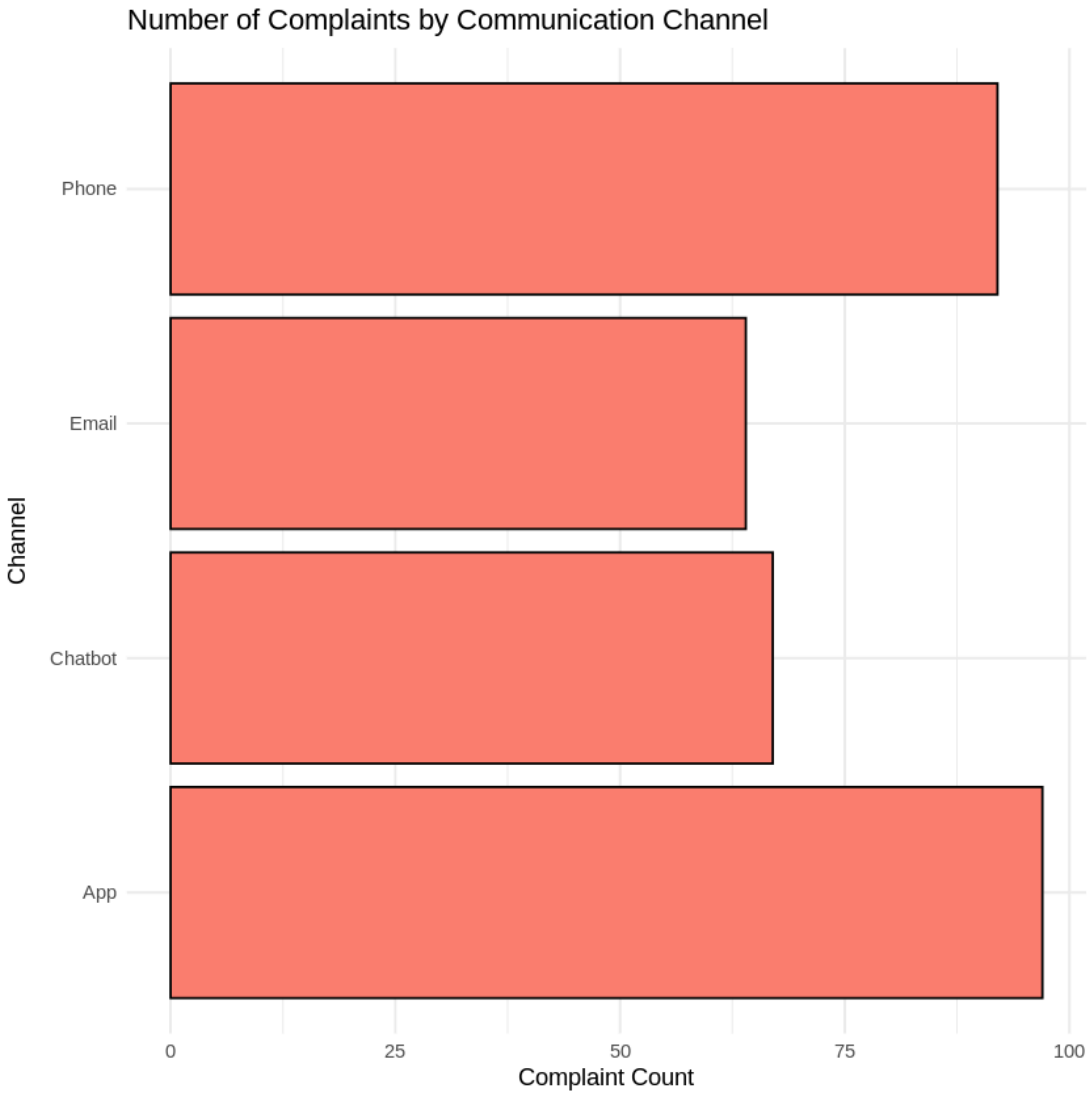

Figure 18: Number of Complaints by Communication Channel

This visualisation analyses the communication channels customers use when submitting complaints. The chart indicates that phone and mobile application channels recorded the highest complaint volumes, providing insight into customer interaction behaviour and support usage patterns.

The results suggest that customers primarily rely on direct communication methods when reporting operational issues. This highlights the importance of maintaining

efficient phone and app-based support systems to manage customer concerns effectively.

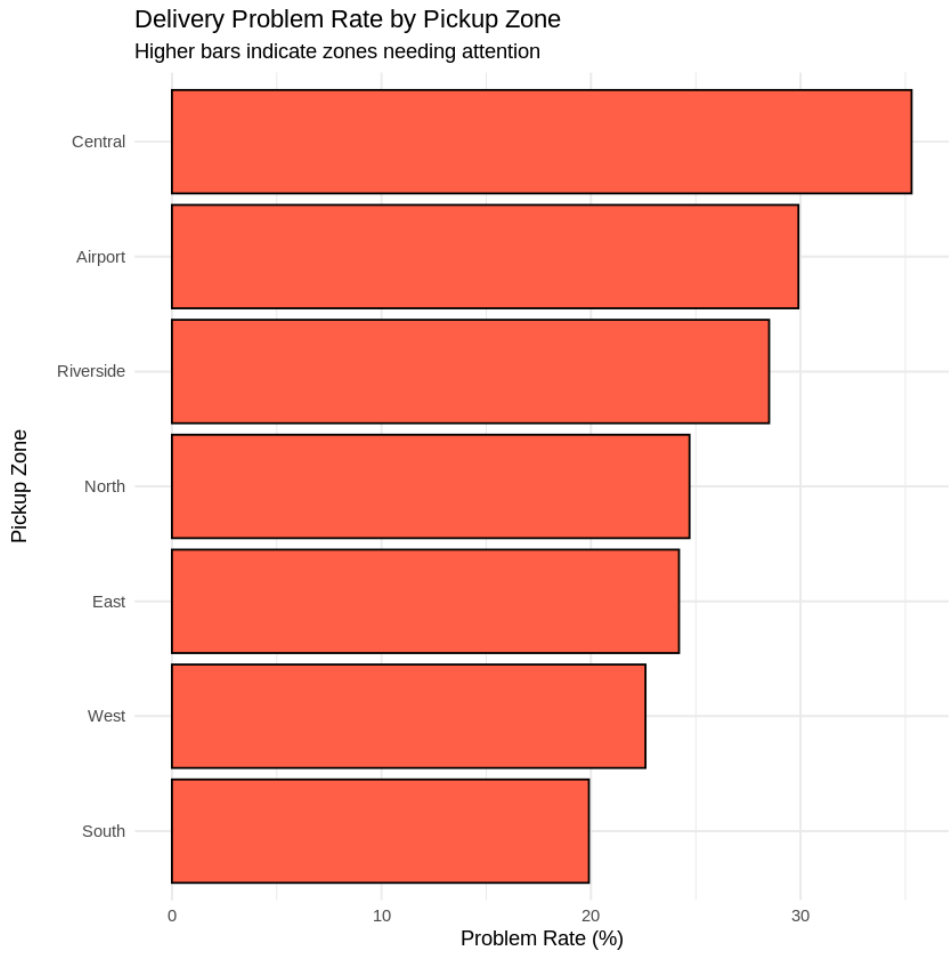

Figure 19: Delivery Problem Rate by Pickup Zone

This figure illustrates the percentage of delivery-related problems across different pickup zones. Furthermore, Central and airport zones recorded the highest operational problem rates, suggesting areas that may require process optimisation, improvements in resource allocation, or operational monitoring.

Higher problem rates in specific zones may indicate traffic congestion, increased operational demand, or logistical inefficiencies. As a result, focusing on operational improvements within these areas could help reduce delivery disruptions and improve Overall service reliability.

Compared with Notebook 2, this notebook is more focused on exploratory reporting and management-facing visualisation. Together, Notebook 2 and Notebook 3 provide both technical query depth and business-readable presentation.

MongoDB Implementation

Notebook 4: 4-NorthStar_MongoDB_Core.ipynb

The fourth notebook demonstrates a NoSQL implementation using MongoDB Atlas. It connects to the northstar_analytics database and loads the same cleaned CSVs, and inserts them into collections using document-oriented modelling. Some collections are built with embedded related data, such as customer records with complaint history and delivery records with order/event context, to support efficient read patterns.

Connection

```
# ------------------------------------------------
# NORTHSTAR URBAN MOBILITY PROJECT
# Notebook 4: MongoDB Database Implementation
#
# This notebook demonstrates the design and implementation
# of a NoSQL database using MongoDB Atlas. It covers data
# loading, document modelling, CRUD operations, indexing,
# and sample analytical queries.
# ------------------------------------------------

!pip install pymongo -q

from pymongo import MongoClient
import pandas as pd
from datetime import datetime

# Establish connection to MongoDB Atlas
client = MongoClient(
    "mongodb+srv://user1:vfg3456dv45ffd@cluster0.icjovks.mongodb.net/?appName=abcCluster",
    tls=True,
    tlsAllowInvalidCertificates=True,
    serverSelectionTimeoutMS=60000
)

db = client["northstar_analytics"]

print("Successfully connected to MongoDB Atlas")
print("Available databases:", client.list_database_names())
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 7.6 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 331.1/331.1 kB 23.9 MB/s eta 0:00:00
Successfully connected to MongoDB Atlas
Available databases: ['northstar_analytics', 'sample_mflix', 'admin', 'local']
```

Figure 20: MongoDB Atlas database connection using PyMongo

This figure demonstrates the successful connection between the Python application and the MongoDB Atlas cloud database using the PyMongo library. The code establishes authentication, connects to the northstar_analytics database, and verifies connectivity by displaying the available databases.

The connection test showed that MongoDB Atlas was working correctly and ready for database operations. The connection test confirmed that MongoDB Atlas was working correctly and could store the project datasets successfully.

Load Data

```
# ------------------------------------------------
# Load All Processed Data Files
#
# These files are the cleaned output from previous notebooks.
# ------------------------------------------------

file_mapping = {
    'complaints': 'complaints_ready.csv',
    'deliveries': 'deliveries_ready.csv',
    'orders': 'orders_ready.csv',
    'vehicles': 'vehicles_ready.csv',
    'drivers': 'drivers_ready.csv',
    'customers': 'customers_ready.csv',
    'incidents': 'incidents_ready.csv',
    'app_events': 'app_events_ready.csv',
    'hubs': 'hubs_ready.csv'
}

loaded_data = {}
for collection_key, filename in file_mapping.items():
    try:
        loaded_data[collection_key] = pd.read_csv(filename)
        print(f"Successfully loaded {collection_key}: {loaded_data[collection_key].shape[0]} rows")
    except Exception as e:
        print(f"Failed to load {filename}: {e}")
```

```
Successfully loaded complaints: 320 rows
Successfully loaded deliveries: 950 rows
Successfully loaded orders: 1250 rows
Successfully loaded vehicles: 120 rows
Successfully loaded drivers: 170 rows
Successfully loaded customers: 650 rows
Successfully loaded incidents: 280 rows
Successfully loaded app_events: 640 rows
Successfully loaded hubs: 8 rows
```

Figure 21: Loading Cleaned Operational Datasets into MongoDB Workflow

This figure illustrates the process of loading cleaned operational datasets into the Python environment before insertion into MongoDB collections. Multiple datasets, including complaints, deliveries, customers, incidents, and vehicles, were successfully imported and prepared for NoSQL database operations.

Successfully loading the datasets ensured that structured operational data could be efficiently transferred into MongoDB collections. This provided the foundation for

document-based storage, querying, and analytical processing within the NoSQL implementation.

Data Upload to Mongo

```
# ------------------------------------------------
# Create Collections with Proper Document Modelling
#
# We use embedding for related data to support efficient reads.
# ------------------------------------------------

# Clear existing collections for clean upload
for coll in ['customers', 'deliveries', 'complaints', 'app_events', 'incidents']:
    if coll in db.list_collection_names():
        db[coll].drop()

# Customers Collection - with embedded complaints
customer_docs = []
for _, row in loaded_data['customers'].iterrows():
    customer_doc = row.to_dict()
    related_complaints = loaded_data['complaints'][loaded_data['complaints']['customer_id'] == customer_doc.get('customer_id')]
    customer_doc['complaint_records'] = related_complaints.to_dict('records')
    customer_docs.append(customer_doc)

db.customers.insert_many(customer_docs)
print(f"Created 'customers' collection with {len(customer_docs)} documents")

# Deliveries Collection - with embedded order and events
delivery_docs = []
for _, row in loaded_data['deliveries'].iterrows():
    delivery_doc = row.to_dict()
    # Embed order details
    order_info = loaded_data['orders'][loaded_data['orders']['order_id'] == delivery_doc.get('order_id')]
    if not order_info.empty:
        delivery_doc['order_info'] = order_info.iloc[0].to_dict()
    # Embed app events
    delivery_doc['event_log'] = loaded_data['app_events'][loaded_data['app_events']['order_id'] == delivery_doc.get('order_id')].to_dict('records')
    delivery_docs.append(delivery_doc)

db.deliveries.insert_many(delivery_docs)
print(f"Created 'deliveries' collection with {len(delivery_docs)} documents")

# Simple collections
db.complaints.insert_many(loaded_data['complaints'].to_dict('records'))
print("Created 'complaints' collection")
```

Figure 22: MongoDB Collection Creation and Data Upload Process

This figure illustrates the creation of MongoDB collections and the upload of operational datasets to the NoSQL database. The implementation includes embedded document modelling, in which related complaint and delivery data are grouped to improve query efficiency and document retrieval performance.

Using embedded documents helped organise related complaint and delivery information more efficiently inside MongoDB; furthermore, this approach improves read performance and supports scalable data management for large business Datasets.

CRUD Operations

```
# ------------------------------------------------
# CRUD Operations Demonstration
# ------------------------------------------------

# READ - Retrieve a sample document
sample = db.deliveries.find_one({"delivery_status": "OnTime"})
print("READ Example - Sample OnTime Delivery:", sample)

# UPDATE - Change status of one open complaint
update_result = db.complaints.update_one(
    {"status": "Open"},
    {"$set": {"status": "In_Progress", "last_updated": datetime.now()}}
)
print("UPDATE Example - Documents modified:", update_result.modified_count)

# DELETE - Safe example (commented to avoid accidental deletion)
# delete_result = db.complaints.delete_one({"complaint_id": "CP9999"})
# print("DELETE Example - Documents deleted:", delete_result.deleted_count)

print("CRUD operations demonstrated")
```

```
READ Example - Sample OnTime Delivery: {'_id': ObjectId('69f6728c0d978446b704d736'), 'delivery_id': 'DL00002', 'order_
UPDATE Example - Documents modified: 1
CRUD operations demonstrated
```

Figure 23: MongoDB CRUD Operations Demonstration

This figure presents the implementation of CRUD (Create, Read, Update, Delete) operations within MongoDB using PyMongo. The code demonstrates retrieving delivery records, updating complaint statuses, and deleting documents from the database to simulate real-world database management operations.

The CRUD operations worked correctly and showed that the MongoDB database could manage operational records effectively. These operations are essential for maintaining accurate, up-to-date, and manageable business records within modern NoSQL systems.

Query Optimisation

Figure 24: Embedded MongoDB Customer Document Structure

This figure presents an example of an embedded MongoDB customer document containing nested complaint records. The document structure stores related operational information, including complaint details, delivery issues, communication channels, and resolution data, within a single document.

The embedded document model demonstrates the flexibility of NoSQL databases for handling related business data. Storing complaint records directly within customer documents improves read efficiency and simplifies the retrieval of connected operational information.

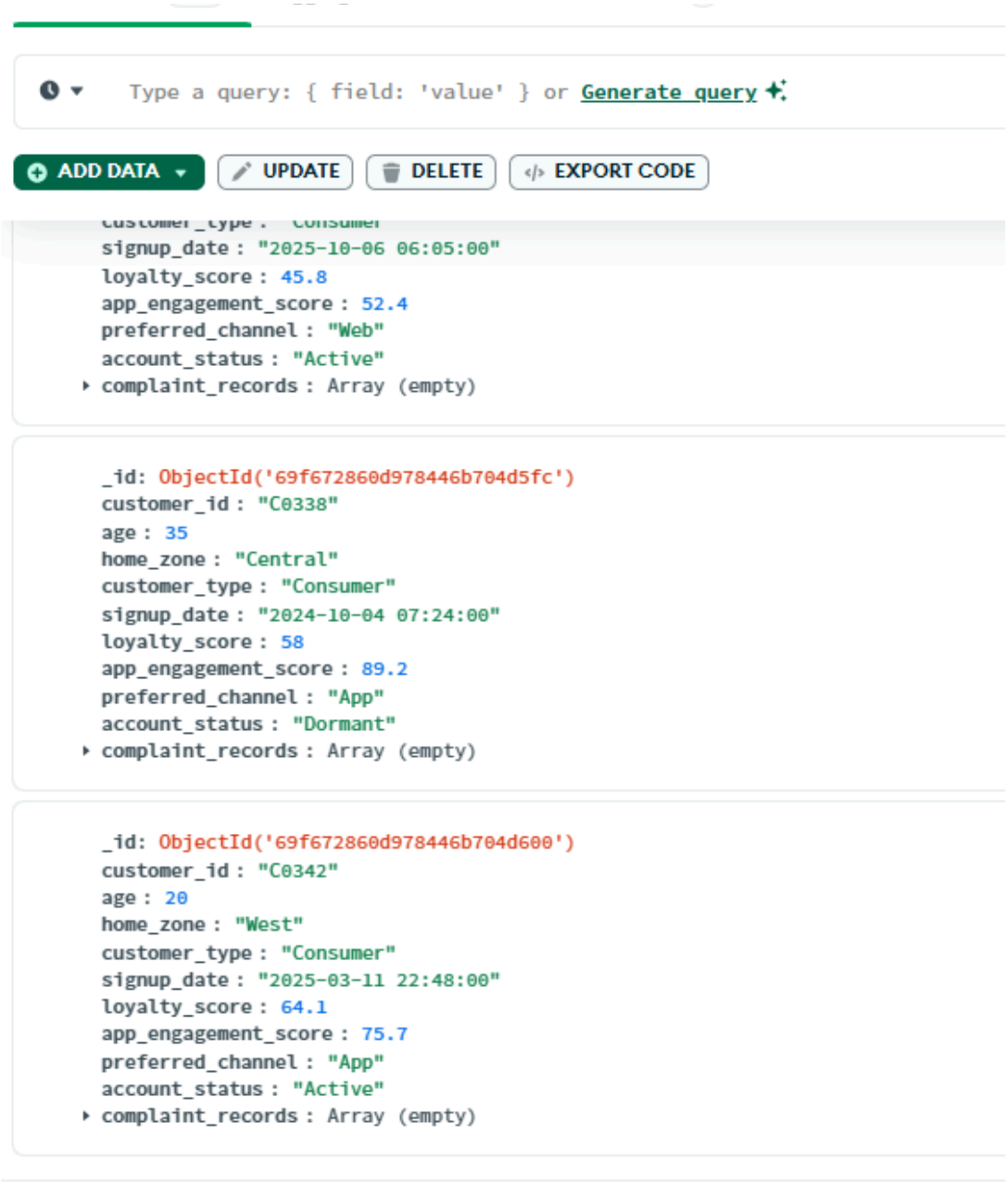

Figure 25: MongoDB Atlas Collection View and Stored Operational Documents

This figure illustrates the MongoDB Atlas collection interface displaying the stored operational documents in the database (MongoDB Inc., 2025). The collection contains

customer-related records, including engagement scores, account statuses, communication preferences, and embedded complaint information.

The successful storage and visualisation of operational documents confirms that the MongoDB implementation was functioning correctly. The document structure made it easier to store different types of operational records inside the database and ensured efficient handling of semi-structured business data.

Findings and Recommendations

1. Improve Route Optimisation and Dispatch Planning

The analysis identified delivery delays and service inconsistencies in busy operational zones such as central and airport areas. NorthStar could improve route planning by using GPS tracking, live traffic updates, and better dispatch scheduling. This may help reduce delays and improve customer satisfaction.

2. Integrate Complaint and Operational Data

Secondly, the Customer complaints, delivery exceptions, and operational records should be integrated into a unified monitoring system. I think combining these datasets would allow management to identify recurring service failures more quickly and improve response times to operational issues.

3. Expand the Use of MongoDB for Semi-Structured Data

MongoDB Atlas should be further adopted for storing flexible and nested operational records such as complaint histories, route exceptions, mobile platform interactions, and event logs. Document-based storage provides greater flexibility for handling evolving operational data compared with rigid relational schemas.

4. Improve Query Performance Through Indexing

Another Recommendation I have is that Indexes should be implemented on high-frequency query fields such as customer_id, delivery_status, zone, and driver_id. This would improve query execution speed, aggregation performance, and overall system scalability as data volumes continue to increase.

5. Create Real-Time Operations Dashboards

I believe another thing NorthStar should do is real-time dashboards to monitor delivery performance, complaint trends, driver incidents, and operational bottlenecks. Visual analytics would support faster decision-making and improve visibility into management across various business areas.

6. Strengthening Predictive and Preventive Analytics

Lastly, the Organisation should further explore predictive analytics techniques to identify risks such as likely delivery failures, recurring maintenance problems, and high-risk operational zones. Early detection of these issues could reduce operational costs and improve service reliability; furthermore, it could help identify drivers or zones with repeated delivery delays before problems become more serious.

End-to-End Workflow Summary

1. **Notebook 1** focuses on cleaning and exporting production-ready CSVs.
2. **Notebook 2** consumes those CSVs for SQL querying in R/SQLite.
3. **Notebook 3** uses those CSVs to generate insights and dashboard-style visuals.
4. **Notebook 4** consumes those CSVs for MongoDB document modelling and NoSQL operations.

The notebooks form a coherent analytics pipeline from data preparation -> relational querying -> business visualisation -> NoSQL implementation.

Critical Evaluation

One limitation of this project is that the datasets were simulated operational datasets rather than real-world live business data. Because of this, some operational patterns may not fully represent real logistics environments.

Github Repositories Link

Conclusion

This project has demonstrated how Python, R, SQL, and MongoDB Atlas can all be combined into a single analytics workflow to address operational and analytical challenges within NorthStar Urban Mobility and Logistics. Raw operational datasets were cleaned and prepared with Python, and structured and business-oriented analysis was queried with SQL in R.

R analytics and visualisation techniques were used to obtain insights on customer complaints, delivery outcomes, operational inefficiencies and service performance patterns.MongoDB Atlas was also implemented as a scalable NoSQL solution for storage and querying of complex, semi-structured and nested operational records(MongoDB Inc., 2025). The CRUD operations, aggregation pipelines, and indexing strategies showed how MongoDB can be utilised to support flexible operational analytics and improve query performance.

The analysis identified a number of key operational problems impacting NorthStar, including delivery delays, inconsistent service, customer dissatisfaction and fragmented data management processes. The project demonstrated that a mix of relational analytics and document-based NoSQL technologies is a more effective approach for handling large-scale operational data and supporting data-driven decision-making.

Overall, the project showed how Python, R, SQL, and MongoDB can work together to analyse operational data and identify service problems and generally improved my practical skills in these modules.

References

MongoDB Inc. (2026) MongoDB Atlas Documentation. Available at: https://www.mongodb.com/docs/atlas/(Accessed: 9 March 2026).

Visual Studio Code(2026). Available at:https://code.visualstudio.com (Accessed: 6 February 2026).

Pandas Development Team (2025) Pandas Documentation.Available at: https://pandas.pydata.org/docs(Accessed: 9 March 2026).

Hardley, Wickham,(2025) ggplot2 Documentation. Available at: https://ggplot2.tidyverse.org/ (Accessed: 3 March 2026).

GitHub (2025) NorthStar Analytics Project Repository. Available at: Accessed: 10 May 2026).

Precision Data Analytics(2024). MongoDB Tutorial for Beginners. YouTube video. Available at: https://youtu.be/AwQmcs1g5js?si=gIEr2_lvcYPK8jye(Accessed: 12 May 2026).